Kubernetes Scenario-Based Questions

Scenario 1: Pod is in CrashLoopBackOff

Scenario

A production application is down. When you check the pods:

```
kubectl get pods
```

Output:

```
NAME                        READY   STATUS             RESTARTS
payment-app                 0/1     CrashLoopBackOff   15
```

The interviewer asks:

- What is CrashLoopBackOff?
- How will you troubleshoot it?
- What are the possible causes?
- How will you fix it?

What is CrashLoopBackOff?

`CrashLoopBackOff` means the container starts, crashes, Kubernetes restarts it, and after repeated failures, Kubernetes increases the delay (BackOff) before restarting again.

Flow:

```
Container Starts
      ↓
Application Crashes
      ↓
Container Exits
      ↓
kubelet Restarts Container
      ↓
Crashes Again
      ↓
Exponential BackOff
```

Step-by-Step Troubleshooting

Step 1: Check Pod Status

```
kubectl get pods -A
```

Verify:

- Restart count
- Status
- Namespace

Step 2: Describe the Pod

```
kubectl describe pod <pod-name>
```

Check:

- Events
- Exit reason
- Probe failures
- Image issues
- Volume mount errors
- OOMKilled

This command usually gives the first clue.

Step 3: Check Logs

```
kubectl logs <pod-name>
```

If the pod restarts quickly:

```
kubectl logs <pod-name> --previous
```

This shows logs from the previous crashed container.

Step 4: Check Recent Changes

If it was working earlier, verify:

- New deployment
- Image change
- ConfigMap update
- Secret update

```
kubectl rollout history deployment <deployment-name>
```

Step 5: Check Resources

```
kubectl top pod
kubectl top node
```

Look for:

- High CPU
- High Memory
- OOMKilled

Common Root Causes

Cause	How to Identify	Solution
Application Bug	Application exception in logs	Fix application code
Wrong Environment Variables	Incorrect DB/API values	Update Deployment/ConfigMap
Database Connection Failure	`Connection Refused` in logs	Verify DB connectivity
Missing ConfigMap/Secret	Events show missing resource	Recreate ConfigMap/Secret
Wrong Docker Image/ENTRYPOINT	`exec: no such file`	Build correct image
Liveness Probe Failure	`Liveness probe failed`	Correct probe configuration
Startup Probe Missing	App takes long to start	Add `startupProbe`
OOMKilled	`Reason: OOMKilled`	Increase memory limits
Volume/PVC Issue	Mount failure in Events	Fix PVC/Storage
Permission Issue	`Permission denied`	Update SecurityContext

Important Commands

```
kubectl get pods -A

kubectl describe pod <pod-name>

kubectl logs <pod-name>

kubectl logs <pod-name> --previous

kubectl top pod

kubectl top node

kubectl rollout history deployment <deployment-name>
```

Real Production Example

A Java application started taking **90 seconds** to initialize after a new release.

The liveness probe was configured to check after **15 seconds**, so Kubernetes kept killing the container before startup completed, causing `CrashLoopBackOff`.

Solution:

- Added a `startupProbe`
- Increased `initialDelaySeconds` for the liveness probe

The pods started successfully without changing the application code.

Best Practices

- Configure `startupProbe` for slow-starting applications.
- Set proper CPU and memory requests/limits.
- Validate ConfigMaps and Secrets before deployment.
- Use meaningful health endpoints.
- Monitor restart counts with Prometheus/Grafana.
- Test deployments in staging before production.

Interview Tips

Q. Which command do you run first?

Answer: `kubectl describe pod <pod-name>` because it provides events, restart reason, probe failures, image issues, and mount errors.

Q. If there are no logs?

Use:

```
kubectl logs <pod-name> --previous
```

Then check:

- Events
- Exit code
- Image

- ENTRYPOINT
- ConfigMaps
- Secrets
- Resource limits

Q. Difference between `CrashLoopBackOff` and `Error`?

- **Error:** Container has stopped due to an error.
- **CrashLoopBackOff:** Kubernetes keeps restarting the crashed container with increasing delays.

Scenario 2: Pod Stuck in Pending

Scenario

A developer deploys an application, but the pod remains in the **Pending** state.

```
kubectl get pods
```

Output:

```
NAME          READY   STATUS    AGE
payment-app   0/1     Pending   10m
```

Interview Questions

- What does Pending mean?
- How will you troubleshoot it?
- What are the common causes?
- How do you fix it?

What is Pending?

A Pod is in **Pending** when it has been created but **Kubernetes cannot schedule it onto any worker node**.

Flow:

```
API Server
     ↓
Scheduler
     ↓
No suitable Node
     ↓
Pod remains Pending
```

Troubleshooting Steps

1. Check Pod Status

```
kubectl get pods -A
```

---

2. Describe the Pod (Most Important)

```
kubectl describe pod <pod-name>
```

Check the **Events** section.

Example:

```
Insufficient memory
```

or

```
node(s) had taints that the pod didn't tolerate
```

---

3. Check Nodes

```
kubectl get nodes
kubectl describe node <node-name>
```

Verify:

- Node is Ready
- Available CPU/Memory
- No Disk/Memory Pressure

---

4. Check Resource Requests

```
kubectl top nodes
```

If the pod requests more resources than any node can provide, it remains Pending.

---

5. Check PVC

```
kubectl get pvc
```

If the PVC is Pending, the pod cannot start.

Common Causes

Cause	Solution
Insufficient CPU/Memory	Add nodes or reduce requests
Node NotReady	Recover the node
Taints/Tolerations	Add toleration or use another node
NodeSelector/Affinity Mismatch	Correct labels/affinity
PVC Pending	Fix PV/StorageClass
StorageClass Missing	Create StorageClass
Cluster Full	Enable/Add Cluster Autoscaler

Important Commands

```
kubectl get pods

kubectl describe pod <pod>

kubectl get nodes

kubectl describe node <node>

kubectl top nodes

kubectl get pvc

kubectl get events --sort-by=.metadata.creationTimestamp
```

Real Production Example

A deployment requested:

```
cpu: 8
memory: 32Gi
```

All worker nodes had only **4 CPU and 16Gi RAM**.

Scheduler Event:

```
0/5 nodes available:
Insufficient CPU
Insufficient Memory
```

Solution:

- Reduced resource requests.
- Added new worker nodes using Cluster Autoscaler.

Best Practices

- Set realistic CPU/Memory requests.
- Enable Cluster Autoscaler.
- Monitor node utilization.
- Verify labels, taints, and StorageClasses before deployment.

Interview Tips

Q. Which command do you run first?

```
kubectl describe pod <pod-name>
```

The **Events** section usually identifies the root cause.

Q. Can a Pending pod have logs?

No. The container has not started yet.

Q. What is the difference between Pending and ContainerCreating?

- **Pending:** Pod is not scheduled.
- **ContainerCreating:** Pod is scheduled, and the kubelet is creating the container.

Scenario 3: Worker Node Becomes NotReady

Scenario

One of your worker nodes suddenly becomes **NotReady**, and some applications become unavailable.

Interview Questions

- How will you troubleshoot?
- What happens to the pods?

- How do you recover the node?

What is NotReady?

The node stops sending heartbeats to the control plane, so Kubernetes marks it as **NotReady**.

Flow:

```
Worker Node
     ↓
Heartbeat Stops
     ↓
Node Controller
     ↓
Node = NotReady
     ↓
Pods are Evicted (after timeout)
```

Troubleshooting

```
kubectl get nodes

kubectl describe node <node>

kubectl get pods -o wide

kubectl describe pod <pod>
```

Check:

- kubelet status
- Container runtime (containerd/Docker)
- CPU/Memory/Disk usage
- Network connectivity
- Node events

Common Causes

Cause	Solution
kubelet stopped	Restart kubelet
Container runtime down	Restart containerd
Disk full	Clean disk
Network issue	Restore connectivity
Node crash	Reboot/replace node

Production Example

A worker node disk became 100% full. kubelet stopped reporting heartbeats, the node became **NotReady**, and Kubernetes rescheduled workloads to healthy nodes after draining the node.

Scenario 4: Application Downtime During Rolling Update

Scenario

A deployment completed successfully, but users experienced downtime.

Interview Questions

- Why did downtime occur?
- How do you ensure zero-downtime deployment?

Troubleshooting

Check rollout:

```
kubectl rollout status deployment <deployment>

kubectl describe deployment <deployment>

kubectl get rs
```

Verify:

- Readiness Probe
- maxUnavailable
- maxSurge
- Pod status

Common Causes

Cause	Solution
Readiness Probe missing	Configure readiness probe
maxUnavailable too high	Reduce value
Single replica	Run multiple replicas
Application startup delay	Use startupProbe

Best Practices

- Minimum 2 replicas
- Proper readiness probe
- RollingUpdate strategy
- PodDisruptionBudget

Production Example

During deployment, traffic was sent to pods before the application finished initializing because no readiness probe was configured. Adding the readiness probe eliminated downtime.

Scenario 5: High CPU Usage in Production

Scenario

Users report slow response times. Monitoring shows pod CPU usage above 90%.

Interview Questions

- How do you investigate?
- Should you increase CPU or replicas?
- Difference between HPA and VPA?

Troubleshooting

```
kubectl top pod

kubectl top node

kubectl describe pod <pod>
```

Check:

- CPU utilization
- Resource requests/limits
- HPA status
- Application logs

Common Causes

Cause	Solution
High traffic	Scale using HPA
CPU limits too low	Increase limits
Inefficient code	Optimize application
Infinite loop	Fix application

Important Concepts

- **HPA:** Scales pods horizontally based on CPU/Memory.
- **VPA:** Adjusts CPU/Memory requests and limits.

- **Cluster Autoscaler:** Adds/removes worker nodes.

Production Example

A flash sale caused CPU usage to reach 95%. HPA automatically increased replicas from 4 to 12, maintaining application availability.

Scenario 6: Pods Restarting Due to OOMKilled

Scenario

Pods restart frequently, and the reason is **OOMKilled**.

Interview Questions

- What is OOMKilled?
- How do you resolve it?

Troubleshooting

```
kubectl describe pod <pod>
```

```
kubectl top pod
```

Look for:

```
Reason: OOMKilled
```

Common Causes

Cause	Solution
Memory limit too low	Increase memory
Memory leak	Fix application
Incorrect JVM heap	Tune heap size

Best Practices

- Set realistic memory requests and limits.
- Monitor memory with Prometheus/Grafana.
- Test memory usage before production.

Production Example

A Java application exceeded its 512Mi limit because the JVM heap was set to 1Gi. After reducing the heap size and increasing the pod memory limit, restarts stopped.

Scenario 7: Service Not Reachable

Scenario

Pods are **Running**, but the application cannot be accessed through the Kubernetes Service.

Interview Questions

- Where will you troubleshoot?
- Why are endpoints empty?
- How does kube-proxy help?

Troubleshooting

```
kubectl get svc

kubectl describe svc <service>

kubectl get endpoints

kubectl get pods --show-labels
```

Verify:

- Service selector
- Pod labels
- Endpoints
- TargetPort
- ContainerPort

Common Causes

Cause	Solution
Wrong selector	Match pod labels
Wrong targetPort	Correct Service definition
Pods not Ready	Fix readiness probe
Network Policy	Allow traffic

Production Example

Pods had the label:

```
app: payment-v2
```

The Service selector expected:

```
app: payment
```

No endpoints were created, so traffic never reached the pods. Updating the selector immediately restored service.

Quick Revision (Scenarios 3–7)

Scenario	First Command	Most Common Root Cause
Node NotReady	`kubectl describe node`	kubelet/runtime failure
Deployment Downtime	`kubectl rollout status`	Missing readiness probe
High CPU	`kubectl top pod`	Traffic spike or low CPU limits
OOMKilled	`kubectl describe pod`	Memory limit exceeded
Service Not Reachable	`kubectl get endpoints`	Selector/targetPort mismatch

Scenario 8: Ingress Returning 503 Service Unavailable

Scenario

The application is accessible inside the cluster, but users accessing it through the Ingress receive:

```
503 Service Unavailable
```

Interview Questions

- How do you troubleshoot?
- Is the problem with Ingress or the application?
- What components will you check?

Troubleshooting

```
kubectl get ingress
```

```
kubectl describe ingress <ingress-name>
```

```
kubectl get svc
```

```
kubectl get endpoints
```

```
kubectl logs -n ingress-nginx <ingress-controller-pod>
```

Check:

- Ingress rules
- Backend Service
- Endpoints
- DNS
- TLS certificate
- Ingress Controller logs

Common Causes

Cause	Solution
Service has no endpoints	Fix Service selector
Wrong backend service	Update Ingress
Pods not Ready	Fix readiness probe
TLS misconfiguration	Correct certificate/secret
Ingress Controller issue	Restart/check logs

Production Example

The Service selector didn't match the pod labels, so the Service had **no endpoints**. The Ingress returned **503** until the selector was corrected.

Scenario 9: ImagePullBackOff

Scenario

A deployment is created, but the pod status shows:

```
ImagePullBackOff
```

Interview Questions

- Why can't Kubernetes pull the image?
- How do you troubleshoot?
- What if it's a private registry?

Troubleshooting

```
kubectl describe pod <pod>
```

```
kubectl get secrets
```

```
kubectl describe secret <image-pull-secret>
```

Check:

- Image name/tag
- Registry access
- ImagePullSecret
- Network connectivity

Common Causes

Cause	Solution
Wrong image name/tag	Correct image reference
Image doesn't exist	Push image
Private registry authentication	Configure ImagePullSecret
ECR token expired	Refresh credentials
Registry unavailable	Check connectivity

Production Example

A deployment referenced image tag **v2.5**, but only **v2.4** existed in ECR. Updating the Deployment resolved the issue.

Scenario 10: Stateful Application Lost Data

Scenario

A database pod restarted, and all application data disappeared.

Interview Questions

- Why did the data disappear?
- Why shouldn't databases use Deployments?
- What is StatefulSet?

Troubleshooting

```
kubectl get pvc
```

```
kubectl get pv
```

```
kubectl describe pvc
```

Verify:

- PVC status
- StorageClass
- Volume mounts

Common Causes

Cause	Solution
Using Deployment instead of StatefulSet	Use StatefulSet
No Persistent Volume	Create PVC/PV
Wrong reclaim policy	Use Retain
Incorrect volume mount	Fix mount path

Important Concepts

- **Deployment** → Stateless applications
- **StatefulSet** → Databases and stateful applications
- **PVC** → Persistent storage
- **Headless Service** → Stable network identity

Production Example

A MySQL database was deployed as a Deployment with `emptyDir`. When the pod restarted, all data was lost. Migrating to a StatefulSet with a PVC solved the problem.

Scenario 11: Rollout Failed After Deployment

Scenario

After deploying a new version, half the pods are running the old version, and half the new version. Users report inconsistent behavior.

Interview Questions

- How do you rollback?
- How do you verify rollout status?
- What is a ReplicaSet?

Troubleshooting

```
kubectl rollout status deployment <deployment>

kubectl rollout history deployment <deployment>

kubectl get rs
```

Rollback:

```
kubectl rollout undo deployment <deployment>
```

Common Causes

Cause	Solution
Application bug	Rollback deployment
Failed readiness probe	Fix application/probe
Image issue	Deploy correct image
Incorrect configuration	Update ConfigMap/Secret

Production Example

Version **v2** failed health checks during deployment. Kubernetes paused the rollout. The team rolled back to **v1** using:

```
kubectl rollout undo deployment payment
```

Scenario 12: Pods Cannot Resolve DNS

Scenario

Application A cannot communicate with Application B using the Service name.

Example:

```
nslookup payment-service
```

returns:

```
Name not found
```

Interview Questions

- How does Kubernetes DNS work?
- What do you check first?
- How do you troubleshoot CoreDNS?

Troubleshooting

```
kubectl get svc
```

```
kubectl get pods -n kube-system
```

```
kubectl logs -n kube-system <coredns-pod>
```

```
kubectl exec -it <pod> -- nslookup payment-service
```

Check:

- CoreDNS
- Service exists
- Namespace
- Network Policies

Common Causes

Cause	Solution
CoreDNS down	Restart CoreDNS
Wrong Service name	Use correct DNS name
Wrong namespace	Use FQDN
Network Policy blocking DNS	Allow DNS traffic

Production Example

An application in the **dev** namespace tried accessing a Service in the **prod** namespace using only the short Service name. Using the full DNS name:

```
payment-service.prod.svc.cluster.local
```

resolved the issue.

Quick Revision (Scenarios 8–12)

Scenario	First Command	Most Common Root Cause
Ingress 503	`kubectl describe ingress`	No backend endpoints
ImagePullBackOff	`kubectl describe pod`	Wrong image/ImagePullSecret
Database Lost Data	`kubectl get pvc`	No persistent storage
Rollout Failed	`kubectl rollout status`	Bad deployment/image
DNS Failure	`kubectl logs coredns`	CoreDNS or namespace issue

Frequently Asked Follow-up Questions

Q1. Difference between `ErrImagePull` and `ImagePullBackOff`?

- **ErrImagePull:** First image pull attempt failed.
- **ImagePullBackOff:** Kubernetes keeps retrying with exponential backoff.

Q2. Why use StatefulSet instead of Deployment for databases?

Because StatefulSet provides:

- Stable pod names
- Persistent storage
- Ordered deployment and termination
- Stable network identity

Q3. How do you rollback a failed deployment?

```
kubectl rollout undo deployment <deployment>
```

Q4. How do you verify a Service is routing traffic correctly?

```
kubectl get endpoints <service-name>
```

If the endpoints list is empty, the Service has no healthy backend pods.

Q5. How do you test DNS from inside a pod?

```
kubectl exec -it <pod> -- nslookup <service-name>
```

or

```
kubectl exec -it <pod> -- curl http://<service-name>:<port>
```

Scenario 13: PVC Stuck in Pending

Scenario

A pod is not starting because its PVC remains in the **Pending** state.

Interview Questions

- Why is the PVC Pending?
- How do you troubleshoot it?
- What is the role of StorageClass?

Troubleshooting

```
kubectl get pvc
```

```
kubectl describe pvc <pvc-name>
```

```
kubectl get pv
```

```
kubectl get storageclass
```

Check:

- StorageClass exists
- Available PV
- Access mode (RWO/RWX)
- CSI Driver status

Common Causes

Cause	Solution
No StorageClass	Create StorageClass
No PV available	Create PV or enable dynamic provisioning
Access mode mismatch	Use correct access mode
CSI Driver issue	Restart/fix CSI Driver

Production Example

A new EKS cluster didn't have the **EBS CSI Driver** installed. PVCs remained Pending until the CSI driver was installed.

Scenario 14: Node Running Out of Disk Space

Scenario

Applications suddenly restart, and one worker node shows **DiskPressure**.

Interview Questions

- What happens when a node runs out of disk?
- How do you recover it?

Troubleshooting

```
kubectl describe node <node>
```

```
kubectl top node
```

SSH into the node:

```
df -h
```

```
du -sh /var/lib/containerd/*
```

Check:

- Container images
- Container logs
- Disk usage
- Image garbage collection

Common Causes

Cause	Solution
Old container images	Remove unused images
Large logs	Rotate/delete logs
Application writing excessive data	Fix application
Small disk size	Expand disk

Production Example

Container images accumulated over several months, filling the disk. Cleaning unused images and enabling image garbage collection resolved the issue.

Scenario 15: Schedule Pods Only on GPU Nodes

Scenario

Machine Learning workloads should run **only on GPU nodes**, while normal applications should never be scheduled there.

Interview Questions

- How do you achieve this?
- Difference between NodeSelector, Affinity, and Taints?

Solution

Label GPU node:

```
kubectl label node worker-1 gpu=true
```

Deployment:

```
nodeSelector:
  gpu: "true"
```

Protect GPU node:

```
kubectl taint nodes worker-1 gpu=true:NoSchedule
```

Add toleration only for ML workloads.

Concepts

Feature	Purpose
NodeSelector	Simple node selection
Node Affinity	Advanced scheduling rules
Taints	Block unwanted pods
Tolerations	Allow selected pods

Production Example

GPU nodes were reserved for AI workloads using **taints and tolerations**, preventing general applications from consuming expensive GPU resources.

Scenario 16: Production Kubernetes Cluster Upgrade

Scenario

Your EKS cluster needs to be upgraded from **1.29 to 1.30** with **zero downtime**.

Interview Questions

- What is the upgrade sequence?
- How do you minimize downtime?
- What should you verify before upgrading?

Upgrade Steps

1. Review Kubernetes version compatibility.
2. Take backups (etcd for self-managed clusters).
3. Upgrade Control Plane.
4. Upgrade managed node groups.
5. Upgrade add-ons:
    - CoreDNS
    - kube-proxy

    - EBS CSI Driver
6. Drain old nodes:

```
kubectl drain <node> --ignore-daemonsets
```

7. Validate workloads.

Common Causes of Upgrade Failure

Cause	Solution
Deprecated APIs	Update manifests
Old add-ons	Upgrade add-ons
Incompatible applications	Test in staging first

Production Example

During an EKS upgrade, CoreDNS wasn't upgraded, causing DNS failures. Updating CoreDNS resolved the issue.

Scenario 17: Network Policy Blocking Traffic

Scenario

Application A cannot communicate with Application B, although both pods are Running.

Interview Questions

- How do you troubleshoot?
- What is a NetworkPolicy?
- How do you verify connectivity?

Troubleshooting

```
kubectl get networkpolicy
```

```
kubectl describe networkpolicy
```

```
kubectl exec -it <pod> -- curl http://service
```

Check:

- Ingress rules
- Egress rules
- Namespace selector

- Pod selector

Common Causes

Cause	Solution
Ingress blocked	Update policy
Egress blocked	Allow outbound traffic
Wrong labels	Fix selectors
CNI issue	Verify CNI plugin

Production Example

A new NetworkPolicy denied all ingress traffic except from the **frontend** namespace. Backend services became inaccessible until the policy was updated.

Quick Revision (Scenarios 13–17)

Scenario	First Command	Common Root Cause
PVC Pending	`kubectl describe pvc`	Missing StorageClass/CSI
DiskPressure	`kubectl describe node`	Disk full
GPU Scheduling	`kubectl label node`	Missing labels/taints
Cluster Upgrade	`kubectl get nodes`	Add-on incompatibility
Network Policy	`kubectl get networkpolicy`	Incorrect ingress/egress rules

Frequently Asked Follow-up Questions

Q1. Difference between PV and PVC?

- **PV (Persistent Volume):** Actual storage resource.
- **PVC (Persistent Volume Claim):** Request for storage made by a pod.

Q2. Why use Taints instead of NodeSelector?

- **NodeSelector** tells Kubernetes where a pod *can* run.
- **Taints** prevent other pods from being scheduled unless they have a matching toleration.

Q3. Why upgrade add-ons after the control plane?

Add-ons such as **CoreDNS**, **kube-proxy**, and **CSI Drivers** must match the Kubernetes version to ensure networking and storage continue to work correctly.

Q4. How do you safely upgrade worker nodes?

```
kubectl drain <node>

Upgrade node

kubectl uncordon <node>
```

Repeat one node at a time to avoid downtime.

Q5. What happens if no NetworkPolicy exists?

By default, **all pod-to-pod traffic is allowed** (unless restricted by the CNI implementation or cloud firewall rules).

Scenario 18: Secret Updated but Application Still Uses Old Password

Scenario

The database password was updated in Kubernetes Secret, but the application still uses the old password.

Interview Questions

- Why didn't the application pick up the new Secret?
- How do you update Secrets without downtime?

Troubleshooting

```
kubectl get secret

kubectl describe secret <secret>

kubectl describe pod <pod>
```

Check:

- How Secret is mounted (Environment Variable or Volume)
- Pod restart status

Common Causes

Cause	Solution
Secret used as Environment Variable	Restart Pod/Rollout Restart
Application caches Secret	Restart application
Secret not mounted correctly	Update Deployment

Important Concept

Secret Usage	Behavior
Environment Variable	Requires Pod restart
Volume Mount	File updates automatically (application must reload it)

Production Example

The DB password was rotated. The Secret updated successfully, but the application consumed it through environment variables. A rollout restart loaded the new password.

```
kubectl rollout restart deployment payment-app
```

Scenario 19: Multi-Tenant Kubernetes Cluster

Scenario

A single Kubernetes cluster is shared by multiple development teams.

Interview Questions

- How do you isolate teams?
- How do you prevent one team from affecting another?

Solution

Use:

- Namespace
- RBAC
- ResourceQuota
- LimitRange
- NetworkPolicy
- ServiceAccount

Common Configuration

Requirement	Kubernetes Feature
Team Isolation	Namespace
Access Control	RBAC
CPU/Memory Limits	ResourceQuota
Default Requests/Limits	LimitRange
Network Isolation	NetworkPolicy

Production Example

The Finance and HR teams shared one cluster. Separate namespaces, RBAC roles, and ResourceQuotas prevented unauthorized access and resource exhaustion.

Scenario 20: Complete Production Incident (Most Asked)

Scenario

At **2:00 AM**, monitoring reports:

- Website Down
- High CPU
- Multiple Pods Restarting
- One Worker Node NotReady
- ALB Health Checks Failing

Interview Question

Walk me through your troubleshooting process from start to finish.

Step-by-Step Troubleshooting

1. Check Cluster Health

```
kubectl get nodes

kubectl get pods -A
```

---

2. Identify Failed Pods

```
kubectl describe pod <pod>

kubectl logs <pod> --previous
```

---

3. Check Node Health

```
kubectl describe node <node>

kubectl top node
```

Look for:

- NotReady
- DiskPressure
- MemoryPressure

---

4. Verify Deployment

```
kubectl rollout status deployment

kubectl rollout history deployment
```

Check if a recent deployment caused the issue.

---

5. Check Service & Endpoints

```
kubectl get svc

kubectl get endpoints
```

Ensure the Service has healthy backend pods.

---

6. Check Ingress / Load Balancer

```
kubectl describe ingress
```

Verify:

- Backend Service
- Target Groups
- Health Checks

---

7. Rollback if Needed

```
kubectl rollout undo deployment <deployment>
```

---

8. Restore Service

- Replace failed node if required.
- Scale application if traffic increased.
- Verify monitoring dashboards.
- Confirm application health.

Production Troubleshooting Flow

```
Alert Received
      ↓
Check Nodes
      ↓
Check Pods
      ↓
Describe Pod
      ↓
Check Logs
      ↓
Check Deployment
      ↓
Check Service
      ↓
Check Ingress
      ↓
Rollback (If Required)
      ↓
Restore Application
      ↓
Perform RCA
```

Common Root Causes

Issue	Resolution
Bad Deployment	Rollback
High CPU	Scale using HPA
Node Failure	Replace/Recover node
OOMKilled	Increase memory
Wrong Service Selector	Fix selector
Readiness Probe Failure	Correct probe
DNS Failure	Restart CoreDNS

Production Example

A new application version contained a memory leak.

Result:

- Pods became **OOMKilled**
- ALB marked pods unhealthy
- Users experienced downtime

Resolution

- Rolled back deployment.
- Increased memory temporarily.
- Developers fixed the memory leak.
- Performed Root Cause Analysis (RCA).

Quick Revision (Scenarios 18–20)

Scenario	First Command	Common Root Cause
Secret Rotation	`kubectl describe secret`	Pod not restarted
Multi-Tenant Cluster	`kubectl get ns`	Missing RBAC/Quotas
Production Incident	`kubectl get nodes`	Deployment/Infrastructure issue

Bonus Scenarios (Very Frequently Asked)

These are also commonly asked in interviews, especially by companies like TCS, Deloitte, Accenture, Capgemini, Cognizant, IBM, EPAM, and product-based companies.

21. Pod Stuck in ContainerCreating

Topics: CNI, CSI, Image Pull, Volume Mount

Commands

```
kubectl describe pod
kubectl get events
```

---

22. One Pod is Slow While Others are Fine

Topics: Node issue, Resource contention, Application logs

Commands

```
kubectl top pod
kubectl describe pod
kubectl top node
```

---

23. HPA Not Scaling Pods

Topics: Metrics Server, Requests, HPA

Commands

```
kubectl get hpa
kubectl top pod
```

---

24. Pods Scheduled on Same Node

Topics: Pod Anti-Affinity, Topology Spread Constraints

Solution

Use:

- Pod Anti-Affinity
- Topology Spread Constraints

to distribute replicas across multiple nodes.

---

25. Worker Node Maintenance

Scenario: Upgrade or patch a worker node without downtime.

Commands

```
kubectl cordon <node>

kubectl drain <node> --ignore-daemonsets

Upgrade Node

kubectl uncordon <node>
```

Final Interview Cheat Sheet

Problem	First Command
CrashLoopBackOff	`kubectl describe pod`
Pending	`kubectl describe pod`
NotReady Node	`kubectl describe node`
High CPU	`kubectl top pod`
OOMKilled	`kubectl describe pod`
Service Issue	`kubectl get endpoints`
Ingress Issue	`kubectl describe ingress`
ImagePullBackOff	`kubectl describe pod`
PVC Pending	`kubectl describe pvc`
DNS Issue	`kubectl logs coredns`
Secret Issue	`kubectl describe secret`
Rollout Failure	`kubectl rollout status`
HPA Issue	`kubectl get hpa`
Network Policy	`kubectl get networkpolicy`
Cluster Upgrade	`kubectl get nodes`

Interview Tip (5–8 Years Experience)